LightGBM 내부 수식 완전 해부

🎯 핵심 목표

> **“Loss를 최소화하기 위해 Gradient와 Hessian을 이용해 트리를 반복적으로 추가하는 모델”**

🧠 1️⃣ 모델 예측식

```
최종 예측 = 모든 트리의 합
\hat{y}i = \sum f_k(x_i)
```}^{K

- f_k = k번째 트리
- K = 트리 개수

2️⃣ 목적 함수 (Objective Function)

```
L = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)
```

항목	의미
l(y, ŷ)	Loss 함수 (예: Binary Log Loss)
Ω(f)	Regularization (트리 복잡도 패널티)

Regularization 예시

```
Ω(f) = γT + 1/2 λ Σ w_j^2
```

- T = leaf 수, w_j = leaf weight, γ = 복잡도 패널티, λ = L2 규제

3️⃣ Gradient Boosting 핵심

Taylor 2차 근사

```
L^{(t)} ≈ Σ_i [g_i f(x_i) + 1/2 h_i f(x_i)^2] + Ω(f)
```

기호	의미
g_i	Gradient (1차 미분)
h_i	Hessian (2차 미분)

정의

```
g_i = ∂l(y_i, ŷ_i)/∂ŷ_i
h_i = ∂^2l(y_i, ŷ_i)/∂ŷ_i^2
```

4️⃣ 트리 학습

Leaf weight 계산

```
w^* = - Σ g_i / (Σ h_i + λ)
```

Split 기준 (Gain)

```
Gain = 1/2 [ (Σ g_L)^2 / (Σ h_L + λ) + (Σ g_R)^2 / (Σ h_R + λ) - (Σ g)^2 / (Σ h + λ) ] - γ
```

- g_L, g_R: 왼쪽/오른쪽 leaf의 gradient 합
- h_L, h_R: Hessian 합
- γ: 복잡도 패널티

5️⃣ Logistic Loss (Binary Classification)

```
p_i = 1 / (1 + exp(-ŷ_i))
```

Gradient & Hessian

```
g_i = p_i - y_i
h_i = p_i * (1 - p_i)
```

- Gradient = 예측 오차
- Hessian = 확신도 / 안정화

6️⃣ 전체 학습 흐름

```
1. 초기 예측 (대충 시작)
2. Gradient 계산 (틀린 방향과 크기)
3. 트리 생성 (패턴 학습)
4. Leaf weight 계산 (Hessian 기반 안정화)
5. 예측 업데이트
6. 반복
```

7️⃣ 금융 관점 적용

- p_i = 부도 확률 (PD)
- Gradient = 모델 오차 (부도/정상 대비)
- Hessian = 안정성/확신도

```
PD 계산 → Gradient로 보정 → Hessian으로 안정화 → Tree로 반복 학습
```

🔥 핵심 요약

- Gradient → 어디로 고칠지 (방향 + 크기)
- Hessian → 얼마나 고칠지 (강도 조절)
- Tree → 어떻게 고칠지 (조건 패턴 학습)

LightGBM = Gradient 기반 오차 보정 + Hessian 안정화 + 트리 반복 학습 시스템